Cycle-Accurate Co-Simulation of Computational Delays in Autonomous Vehicle Control

Author 1*, Author 2†, Author 3*
*Affiliation 1 †Affiliation 2
{email1, email2}@placeholder.edu

***Abstract*—Control algorithms for autonomous vehicles must execute within a strict sample period on embedded processors. When computation exceeds this deadline, the controller misses its update and the vehicle continues under stale commands—a failure mode that is invisible to conventional simulators. We present a co-simulation framework that couples SHARC, a cycle-accurate hardware simulator, with CARLA, a high-fidelity driving simulator, to capture the closed-loop effect of realistic computational delays. The framework models an abstract sampled-data control loop with delay-dependent zero-order hold, then instantiates the plant dynamics through an external simulator. We deploy PID and MPC controllers on parameterized hardware configurations and measure how clock frequency, cache size, and algorithmic complexity jointly determine deadline miss rates, tracking error, and safety. Experiments reveal that MPC controllers exhibit a sharp performance cliff: a small reduction in processor capability can trigger cascading deadline misses and rapid trajectory divergence. PID, by contrast, is computationally inexpensive and immune to this failure mode, though it sacrifices tracking quality. These results demonstrate that hardware and software design for autonomous vehicles must be evaluated jointly, and that the proposed framework provides the necessary simulation fidelity to do so.**

***Index Terms*—Autonomous driving, hardware–software co-design, computational delay, model predictive control, microarchitecture simulation.**

I. Introduction

An autonomous vehicle's control loop seems straightforward: read sensors, compute a command, actuate. The catch is that *computing* the command takes time. On a modern automotive SoC running a PID controller, that time is negligible. On the same chip running nonlinear model predictive control (MPC), it may not be. Shrink the chip—reduce the clock, halve the cache—and the computation may exceed the sample period entirely. When that happens, the vehicle does not receive a fresh command; it continues executing the previous one. In a highway cruise this may be harmless. In an emergency lane change it may be catastrophic.

This coupling between hardware capability and control performance is well understood in principle [1], [2], yet in practice the two domains are designed independently. Control engineers assume instantaneous computation; hardware engineers optimize for generic benchmarks. Integration testing on physical prototypes discovers timing violations late and expensively [3]. What is missing is a simulation tool that evaluates the full loop—plant physics, control algorithm, and processor microarchitecture—in a single experiment.

This paper presents such a tool. We combine two simulators:

- **SHARC** [4] (Simulator for Hardware Architecture and Real-time Control) executes a controller binary on a parameterized processor model via the Scarab cycle-accurate simulator [5], returning the computation time τ_k at every sample step.
- **CARLA** [6], an open-source driving simulator with realistic vehicle dynamics, road networks, and traffic, serves as the plant.

SHARC is designed around an abstract dynamics interface: any function that maps the current state and control to the next state can serve as the plant model. We exploit this by replacing the ODE integrator with CARLA's physics engine, obtaining high-fidelity vehicle dynamics without modifying SHARC's core architecture.

Contributions.

1) A formal sampled-data model of delay-induced control degradation under zero-order hold, independent of any specific plant or controller (Section II).
2) Integration of CARLA as a concrete instantiation of SHARC's abstract dynamics, enabling cycle-accurate delay analysis in realistic driving scenarios (Section III).
3) Experimental characterization of how hardware parameters (clock frequency, cache size) interact with controller complexity (PID vs. MPC) to determine closed-loop safety (Section V).

II. Sampled-Data Control with Computational Delay

We develop a general model of a sampled-data control system in which the controller's computation time is state- and hardware-dependent. This model is independent of any particular plant or simulator; CARLA enters later as one possible instantiation.

A. Plant and Discretization

Consider a continuous-time plant

$$\dot{x} = \tilde{f}(t, x, u, w), \qquad y = h(x), \tag{1}$$

with state $x \in \mathbb{R}^n$, control input $u \in \mathbb{R}^m$, exogenous input $w \in \mathbb{R}^p$, and output $y \in \mathbb{R}^q$. A digital controller samples the

output at times $t_k := kT$, $k \in \mathbb{N}$, with sample period $T > 0$, producing a control update that is applied at the next sample instant under zero-order hold (ZOH). The resulting discrete-time system is

$$x_{k+1} = f(x_k, u_k, w_k), \tag{2}$$

where f encapsulates the continuous dynamics integrated over one period under constant u_k and $w_k := w(t_k)$.

Remark 1 (Generality of f)**.** *Equation* (2) *requires only that f maps the current state, input, and exogenous signal to the next state. The map may be derived analytically, computed by a numerical integrator, or—as in our case—evaluated by an external physics engine. This generality is central to SHARC's design and is what allows us to substitute CARLA as the plant.*

B. Computation Delay and Zero-Order Hold

At each sample time t_k, the controller reads the current output y_k and begins computing a new input $\tilde{u}_{k+1}$. This computation takes $\tau_k > 0$ seconds.

Definition 1 (Sampled Application with ZOH)**.** *Under sampled control with computation delay, the input applied to the plant is*

$$u_k = \begin{cases} \tilde{u}_k & \text{if } \tau_{k-1} < T \quad \textit{(on time)}, \\ u_{k-1} & \text{if } \tau_{k-1} \geq T \quad \textit{(deadline miss)}. \end{cases} \tag{3}$$

That is, the freshly computed input is applied at the next *sample instant if the computation finishes within one period; otherwise, the previous input is held.*

This is the standard one-step delay model used in real-time control systems: the controller computes during period k and the result takes effect at period $k{+}1$. A deadline miss means the new result is discarded and the old input persists.

Definition 2 (Deadline Miss and Miss Rate)**.** *A* deadline miss *at step k occurs when $\tau_k \geq T$. The miss rate over K steps is*

$$\rho := \frac{1}{K} \sum_{k=0}^{K-1} \mathbf{1}[\tau_k \geq T]. \tag{4}$$

C. Hardware-Dependent Computation Time

The computation time is not a fixed constant—it depends on the algorithm, the input data, and the processor:

$$\tau_k = \mathcal{T}(g, y_k, \mathcal{H}), \tag{5}$$

where g is the controller algorithm, y_k the current measurement, and $\mathcal{H} \in \mathbb{R}_+^{n_h}$ is a *hardware configuration* specifying the processor's clock frequency, cache sizes, pipeline depth, and other microarchitectural parameters.

For a PID controller, τ_k is essentially constant regardless of y_k or $\mathcal{H}$: the computation is a fixed sequence of additions and multiplications. For an MPC controller, τ_k depends on the state through the optimization landscape—states far from the reference may require more solver iterations—and on the hardware through the instruction throughput.

Evaluating $\mathcal{T}$ analytically is intractable for nontrivial controllers and realistic processors. SHARC evaluates it empirically by executing the controller binary inside Scarab, a cycle-accurate microarchitecture simulator that models caches, branch predictors, and pipelines at the instruction level.

D. Closed-Loop Dynamics Under Delay

Combining (2) and (3), the closed-loop system under computation delay is

$$x_{k+1} = f\big(x_k,\ \mu(x_{0:k}, \tau_{0:k-1}),\ w_k\big), \tag{6}$$

where μ denotes the effective input policy determined by the history of states and delays through the ZOH rule (3). The key observation is that τ_k itself depends on x_k via (5), creating a feedback loop: the state affects the computation time, which affects the applied input, which affects the next state.

Proposition 1 (Cascading Deadline Misses)**.** *Suppose the computation time is monotonically nondecreasing in the tracking error: $\|x_k - \bar{x}_k\| > \|x_j - \bar{x}_j\|$ implies $\tau_k \geq \tau_j$, where $\bar{x}_k$ is the reference trajectory. If a deadline miss at step k increases the tracking error (i.e., applying the stale input u_{k-1} instead of the fresh $\tilde{u}_k$ moves x_{k+1} further from $\bar{x}_{k+1}$), then step $k{+}1$ is also a deadline miss.*

Proof. By hypothesis, the miss at step k yields $\|x_{k+1} - \bar{x}_{k+1}\| > \|x_k - \bar{x}_k\|$. Since τ is nondecreasing in the error, $\tau_{k+1} \geq \tau_k \geq T$, so step $k{+}1$ is also a miss. The argument repeats inductively. □

This cascade is the mechanism behind the *performance cliff* observed in SHARC experiments: once a controller begins missing deadlines, the resulting tracking errors increase future computation times, which cause further misses. For PID controllers, whose computation time is state-independent ($\tau_k \approx$ const), this cascade cannot occur.

III. Co-Simulation Framework

We now describe the two simulators and how they are integrated.

A. SHARC: Architecture-Aware Control Simulation

SHARC [4] is a framework for evaluating control systems under realistic computational delays. Its architecture consists of three components:

1) **Plant runner.** A Python process that maintains the state x_k, applies the ZOH rule (3), advances the dynamics (2), and logs data.
2) **Controller.** A compiled C/C++ binary that receives (t_k, x_k, w_k) from the plant runner via inter-process communication and returns the control input $\tilde{u}_{k+1}$.
3) **Scarab.** A cycle-accurate microarchitecture simulator that instruments the controller execution and returns the computation time τ_k in simulated clock cycles.

The hardware configuration $\mathcal{H}$ is specified via a parameter file: clock frequency, L1/L2 cache sizes, associativity, memory latency, branch predictor type, and pipeline width. Changing

a single parameter—say, doubling the cache—is a one-line edit, making it straightforward to sweep large regions of the hardware design space.

SHARC provides two Scarab execution modes. In *execution-driven* mode, Scarab simulates the controller instruction-by-instruction during the control loop, producing τ_k in real time. In *trace-driven* mode, the controller first runs natively under DynamoRIO instrumentation to capture memory-access traces, which Scarab then replays offline. The trace-driven mode enables parallel processing of multiple time steps and is significantly faster for long simulations.

Crucially, SHARC's plant dynamics are defined by an abstract interface: any callable $f : \mathbb{R}^n \times \mathbb{R}^m \times \mathbb{R}^p \to \mathbb{R}^n$ can serve as the state transition. The framework imposes no assumptions on the internal structure of f. This is what allows us to replace the default ODE integrator with an external simulator.

B. CARLA: High-Fidelity Driving Simulation

CARLA [6] is an open-source driving simulator built on Unreal Engine. It provides detailed urban environments, realistic vehicle physics (tire models, drivetrain, aerodynamics), traffic participants, and a sensor suite. In *synchronous mode*, the simulation clock advances only when the client issues a tick command, and each tick advances physics by a configurable fixed step Δt.

C. Integration

We implement a concrete dynamics class that connects SHARC's abstract plant interface to CARLA's synchronous API. The class satisfies the following contract:

1) **Initialization.** Connect to the CARLA server, enable synchronous mode with $\Delta t = T$, and spawn the ego vehicle.
2) **State transition.** Given (x_k, u_k, w_k), apply the control input to the ego vehicle, tick the CARLA world, and read back x_{k+1} from the vehicle's transform and velocity.

Because CARLA's physics step Δt is set equal to the sample period T, the discrete-time index k in (2) is exactly aligned with CARLA's simulation tick. The transition function $f_{\text{CARLA}} : \mathbb{R}^n \times \mathbb{R}^m \times \mathbb{R}^p \to \mathbb{R}^n$ is evaluated by CARLA's internal physics engine, which models tire forces, aerodynamic drag, engine torque curves, suspension dynamics, and road geometry. From SHARC's perspective, f_{CARLA} is simply a concrete, high-fidelity instantiation of the abstract f in (2).

Example 1 (State and control representation)**.** *For the experiments in this paper, we use:*

$$x_k := (p_x,\, p_y,\, \psi,\, v,\, w_x^+,\, w_y^+)^\top \in \mathbb{R}^6, \tag{7}$$

$$u_k := (\alpha,\, \delta,\, \beta)^\top \in [0,1] \times [-1,1] \times [0,1], \tag{8}$$

where (p_x, p_y) is position, ψ is yaw, v is speed, (w_x^+, w_y^+) is the next waypoint from the CARLA HD map, and (α, δ, β) are throttle, steering, and brake. The exogenous input $w_k := (w_x^+, w_y^+)^\top$ supplies the waypoint to the controller.

IV. Controller Formulations

We evaluate two controllers with qualitatively different computational profiles. Both are compiled as C++ binaries and instrumented by Scarab.

A. PID Controller

The PID controller decomposes vehicle control into two independent loops.

Longitudinal control tracks a constant reference speed v^*:

$$e_k^v := v^* - v_k, \qquad \sigma_k = K_P^\ell e_k^v + K_I^\ell \sum_{j=0}^{k} e_j^v T + K_D^\ell \frac{e_k^v - e_{k-1}^v}{T}. \tag{9}$$

Throttle and brake are assigned as $\alpha_k = \text{clip}(\sigma_k, 0, 1)$ and $\beta_k = \text{clip}(-\sigma_k, 0, 1)$.

Lateral control steers toward the next waypoint. The heading error is $e_k^\psi := \text{atan2}(w_y^+ - p_y,\, w_x^+ - p_x) - \psi_k$, and the steering command is

$$\delta_k = \text{clip}\big(K_P^r e_k^\psi + K_I^r \textstyle\sum_j e_j^\psi T + K_D^r (e_k^\psi - e_{k-1}^\psi)/T,\, \pm 1\big). \tag{10}$$

PID requires a fixed number of arithmetic operations per step. Its computation time is effectively constant and independent of both the vehicle state and the hardware configuration.

B. Model Predictive Control (MPC)

The MPC controller receives a vector of N_w reference waypoints $\{(w_x^{(i)}, w_y^{(i)})\}_{i=1}^{N_w}$ from the route planner and solves an optimization problem at each sample time.

Prediction model. We use a kinematic bicycle model for internal prediction:

$$\begin{bmatrix} p_x \\ p_y \\ \psi \\ v \end{bmatrix}_{k+1} = \begin{bmatrix} p_x \\ p_y \\ \psi \\ v \end{bmatrix}_{k} + T \begin{bmatrix} v_k \cos\psi_k \\ v_k \sin\psi_k \\ (v_k/L)\tan\delta_k \\ a_k \end{bmatrix}, \tag{11}$$

where L is the wheelbase and a_k is the longitudinal acceleration derived from throttle and brake.

Cost function. The lateral cost penalizes the distance to the *closest* waypoint, promoting path following without requiring explicit waypoint indexing:

$$J = \sum_{j=0}^{N_p-1} \big[q_\ell\, d_j^2 + q_v (v_{k+j} - v^*)^2 + \|u_{k+j}\|_R^2 \big], \tag{12}$$

where $d_j := \min_i \|(p_{x,k+j}, p_{y,k+j}) - (w_x^{(i)}, w_y^{(i)})\|$ is the distance to the closest waypoint, $q_\ell > 0$ weights lateral tracking, $q_v > 0$ weights speed regulation to the reference v^*, and $R \succ 0$ penalizes control effort.

Optimization problem. At each t_k:

$$\min_{u_{k:k+N_c-1}} \quad J \tag{13a}$$

$$\text{s.t.} \quad x_{j+1|k} = F_{\text{bike}}(x_{j|k}, u_{j|k}), \quad j = k, \ldots, k+N_p-1, \tag{13b}$$

$$x_{k|k} = x_k, \tag{13c}$$

$$u_{j|k} \in \mathcal{U}, \quad x_{j|k} \in \mathcal{X}, \tag{13d}$$

with prediction horizon N_p, control horizon $N_c \leq N_p$, and constraint sets $\mathcal{U}$ (actuator limits) and $\mathcal{X}$ (lane boundaries, speed limits). Only the first input $u_{k|k}$ is applied (receding horizon).

Remark 2 (Computation time depends on the optimization landscape)**.** *The number of solver iterations—and hence τ_k—depends on the current state x_k. When the vehicle is near the reference, the optimization is well-conditioned and converges quickly. When the state is perturbed (e.g., after a deadline miss applies a stale input), the solver requires more iterations. This is the mechanism underlying Proposition 1.*

V. Experimental Setup

A. Driving Scenario

All experiments use a lane-following task on CARLA's Town04 highway map. The ego vehicle starts from rest and must track a sequence of waypoints at a reference speed of $v^* = 20$ km/h. NPC traffic vehicles are spawned with autopilot to provide a realistic environment. The simulation runs for K time steps with sample period T.

B. Hardware Configurations

We evaluate each controller on a grid of hardware configurations by varying two parameters in Scarab's configuration file:

- **Clock frequency:** $f_{\text{clk}} \in \{250, 500, 750, 1000, 1500, 2000\}$ MHz, specified via the `chip_cycle_time` parameter (cycle period in femtoseconds).
- **L1 data cache size:** $S_{\text{L1}} \in \{8, 16, 32, 64, 128\}$ kB.

All other microarchitectural parameters (pipeline width, branch predictor, memory latency) are held at a baseline configuration. This yields a $6 \times 5 = 30$-point hardware grid per controller.

C. Delay Scaling

When the raw Scarab-computed delay τ_k^{raw} is much smaller than the sample period (as is typical for simple controllers on fast hardware), deadline misses do not occur and the effect of delay is invisible. To study the full range of delay-induced behavior within a single hardware configuration, we introduce a *delay multiplier* $\lambda \geq 1$:

$$\tau_k = \lambda \cdot \tau_k^{\text{raw}}. \tag{14}$$

The multiplier does not alter the cycle-accurate measurement; it scales the delay that is reported to the plant dynamics. This is equivalent to studying the same controller on a processor that is λ times slower, without re-running Scarab. Both the raw delay τ_k^{raw} and the multiplier λ are recorded in the experiment metadata for reproducibility.

D. Performance Metrics

- **Position RMSE:** $\text{RMSE}_{\text{pos}} := \sqrt{\frac{1}{K}\sum_{k=0}^{K-1}[(p_x^k - \bar{p}_x^k)^2 + (p_y^k - \bar{p}_y^k)^2]}$.
- **Speed RMSE:** $\text{RMSE}_v := \sqrt{\frac{1}{K}\sum_{k=0}^{K-1}(v_k - v^*)^2}$.
- **Deadline miss rate** ρ (Definition 2).
- **Mean computation delay:** $\bar{\tau} := \frac{1}{K}\sum_{k=0}^{K-1}\tau_k$.

E. Execution Modes

SHARC supports two modes of interaction with Scarab:

- **Execution-driven (serial):** Scarab instruments the controller binary at each time step, returning τ_k inline. The closed loop sees the true delay at every step, including any state-dependent variation. This is the most faithful mode.
- **Trace-driven (parallel):** The controller first runs natively under DynamoRIO to capture instruction traces, then Scarab replays each trace offline. Multiple traces are processed in parallel, reducing wall-clock time substantially. When a deadline miss is detected, SHARC re-simulates from the miss point with corrected initial conditions (batching).

VI. Results

[TODO: All experimental results are pending. The subsections below describe the planned analyses and the expected outcomes.]

A. Computation Delay Distributions

[TODO: Box plots of τ_k for PID and MPC across hardware configs. Horizontal line at T. Expected: PID delays well below T on all hardware; MPC delays cross T on constrained configs. Table of $\bar{\tau}$ and ρ.]

TABLE I
Mean delay $\bar{\tau}$ (ms) and miss rate ρ for lane following ($T = 0.1$ s, $\lambda = 1$).

	PID		MPC	
f_{clk} (MHz)	$\bar{\tau}$	ρ	$\bar{\tau}$	ρ
2000	[TODO:]	[TODO:]	[TODO:]	[TODO:]
1000	[TODO:]	[TODO:]	[TODO:]	[TODO:]
500	[TODO:]	[TODO:]	[TODO:]	[TODO:]
250	[TODO:]	[TODO:]	[TODO:]	[TODO:]

B. Tracking Performance vs. Hardware

[TODO: RMSE_{pos} vs. f_{clk} for PID and MPC. Expected: PID flat; MPC shows sharp degradation (cliff) when $\bar{\tau}$ crosses T.]

C. Cascading Miss Dynamics

[TODO: Time series of (τ_k, e_k) for an MPC run that crosses the deadline. Illustrate the positive feedback loop of Proposition 1: miss $\rightarrow$ error growth $\rightarrow$ harder optimization $\rightarrow$ longer τ_{k+1} $\rightarrow$ another miss.]

D. Effect of Delay Multiplier

[TODO: $\mathrm{RMSE}_{\mathbf{pos}}$ vs. λ for MPC on a fixed hardware config. Shows smooth increase followed by cliff when $\lambda\bar{\tau}^{\mathbf{raw}}$ crosses T.]

E. Cache Sensitivity

[TODO: $\mathrm{RMSE}_{\mathbf{pos}}$ vs. $S_{\mathbf{L1}}$ at fixed clock. Expected: PID insensitive; MPC shows improvement with larger cache up to a saturation point.]

VII. Discussion

[TODO: To be written after experimental results. Key points:]

The performance cliff. The most striking finding in SHARC-based experiments—first observed for an inverted pendulum at 600 vs. 625 MHz [4]—is that control degradation is not gradual. A small hardware reduction can flip the system from zero misses to cascading misses. This is fundamentally different from the intuition that "a slower chip gives slightly worse performance."

PID as a robust baseline. Because PID's computation time is state-independent, it cannot enter the cascade of Proposition 1. This makes PID a natural fallback for safety-critical scenarios where hardware guarantees are uncertain.

MPC horizon as a design variable. Reducing the prediction horizon N_p lowers computation time (and hence τ_k) at the cost of tracking quality. The framework enables quantitative evaluation of this tradeoff for a specific chip and scenario, replacing heuristic tuning with data-driven design.

Limitations. CARLA's physics engine, while detailed, is not a certified vehicle model. The delay multiplier captures the *effect* of slower hardware but does not model secondary microarchitectural effects (e.g., thermal throttling). NPC traffic is autopilot-controlled and does not represent adversarial or stochastic agents.

VIII. Conclusion

We presented a co-simulation framework coupling SHARC's cycle-accurate hardware simulation with CARLA's high-fidelity driving physics. The framework instantiates a general sampled-data delay model through CARLA's synchronous API, enabling systematic evaluation of how processor microarchitecture affects closed-loop control safety. Experiments **[TODO: (pending)]** are expected to demonstrate that MPC controllers exhibit a sharp performance cliff under hardware constraints, while PID remains robust at the cost of tracking quality. The proposed framework enables hardware–software co-design for autonomous vehicles before physical prototypes are built.

Future work includes extending the framework to multi-agent scenarios, incorporating perception delays from sensor processing, and using the delay model to design adaptive controllers that degrade gracefully under deadline pressure.

References

[1] M. Caccamo, G. Buttazzo, and L. Sha, "Handling execution overruns in hard real-time control systems," *IEEE Transactions on Computers*, vol. 51, no. 7, pp. 835–849, 2002.

[2] R. Findeisen, L. Grüne, J. Pannek, and P. Varutti, "Robustness of prediction based delay compensation for nonlinear systems," *IFAC Proceedings Volumes*, vol. 44, no. 1, pp. 203–208, 2011.

[3] F. Mihalič, M. Truntič, and A. Hren, "Hardware-in-the-loop simulations: A historical overview of engineering challenges," *Electronics*, vol. 11, no. 15, 2022.

[4] P. K. Wintz, Y. Sonmez, P. Griffioen, M. Xu, S. Oh, H. Litz, R. G. Sanfelice, and M. Arcak, "SHARC: Simulator for Hardware Architecture and Real-time Control," in *Proceedings of the 28th ACM International Conference on Hybrid Systems: Computation and Control (HSCC '25)*. Irvine, CA, USA: ACM, May 2025.

[5] Litz Lab, "Scarab microarchitecture simulator," https://github.com/Litz-Lab/scarab, 2024.

[6] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An Open Urban Driving Simulator," in *Proceedings of the 1st Annual Conference on Robot Learning (CoRL '17)*. PMLR, 2017, pp. 1–16.